資料分析與機器學習 (Data Analysis and Machine Learning)

Lecture 10 –

Regression and Applications

探討「自變數」與「應變數」(連續數值) 之間的預測與因果關係

教師：陳慶永

Outline

- Problem Statement
- Linear Regression
  - Simple linear regression
  - Multiple linear regression
  - Basis function regression
    - Polynomial regression
- Linear Least Squares (LLS) Regression in Python
- Nonlinear Regression
- Classwork
  - Assignment #6

Problem Statement

Problem Statement

- In science and engineering coursework, we typically predict the state (on a continuous scale) of a system using a set of parameters of the system
  - However, in practice, the parameters of a system are not known a priori
  - Instead, we are usually presented with data points about how the system has behaved in the past
    - Data usually has some measurement noise because of sensor inaccuracy, measurement error, etc.

How to find the "most likely" model parameters given a set of noisy data? 🤔

The goal of regression analysis is to model the relationship between one or multiple features and a continuous target variable

Problem Statement

- Regression (回帰)
  - Given a set of independent data points (feature) x_j and dependent data points (target) $y_j,\ j = 1,2,\dots,m,$ we would like to find an **estimation function** $\hat{y}(x)$ that describes the data as well as possible
- *Linear* Regression (線性回帰)
  - Estimation function $\hat{y}(x)$ must be a *linear* combination of **basis functions** $f_i(x),\ i = 0,1,\dots,n-1$

$$\hat{y}(x) = \sum_{i=0}^{n-1} \alpha_i\, f_i(x)$$

    - $\alpha_i,\ i = 0,1,\dots,n-1$ are parameters of the estimation function

$\hat{y}$ can be a function of several variables ($x, t, z,$ etc.) and $\hat{y}(x)$ is an example of single-variable case

Problem Statement

- Least Squares Regression (最小平方法回歸)
  - Find the parameters α_i's of the estimation function that *minimize* the total squared error E defined by

(Loss function) $$E = \sum_{j=1}^{m} |y_j - \hat{y}(x_j)|^2$$

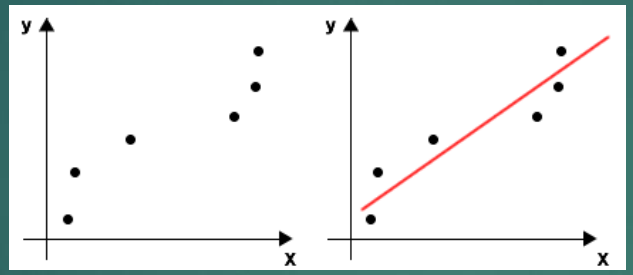
    - Let $e_j = y_j - \hat{y}(x_j)$ denotes individual errors (or offsets or residuals) and $\mathbf{e} = [e_1, e_2, \ldots, e_m]^T$ the error vector 殘差

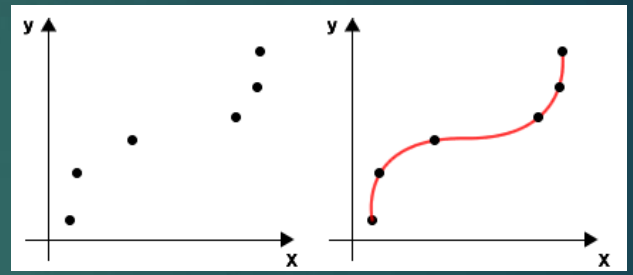
$$E = \|\mathbf{e}\|^2$$

      - $\min_{\alpha_i} E$ is essentially an optimization problem (最佳化問題)

Problem Statement

- Regression v.s. Interpolation





Feature	Regression	Interpolation
Data Point Fit	Does **not** pass through all points	Must pass **exactly** through all points
Primary Goal	Capture the global trend and relationship	Estimate missing values within a known range
Handling Noise	Excellent; smoothes out errors and outliers	Poor; susceptible to overfitting and distortion
Mathematical Goal	Minimize the overall residual error	Solve exact equations for local parameters
Typical Common Use	Finding patterns, physics laws, or machine learning	Increasing sampling resolution, filling gaps

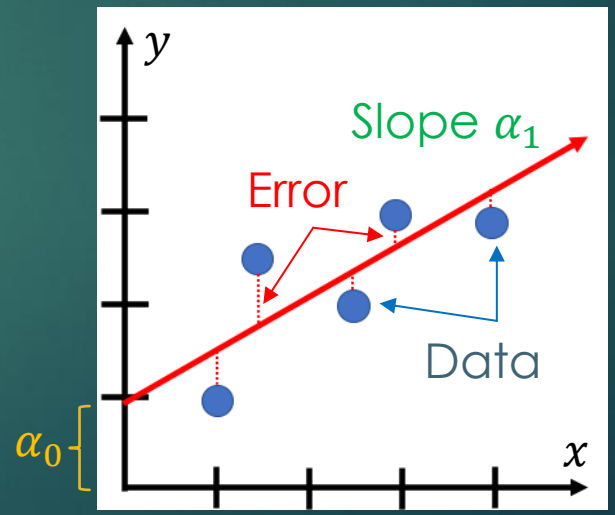
Linear Regression

Linear Regression

- Simple (univariate) Linear Regression (簡單線性回歸)
  - The most familiar linear regression
  - Using a straight-line $(y(x) = ax + b)$ to fit data

$$\hat{y}(x) = \alpha_1 x + \alpha_0$$

  - α_1: Slope (斜率)
  - α_0: Intercept (截距)



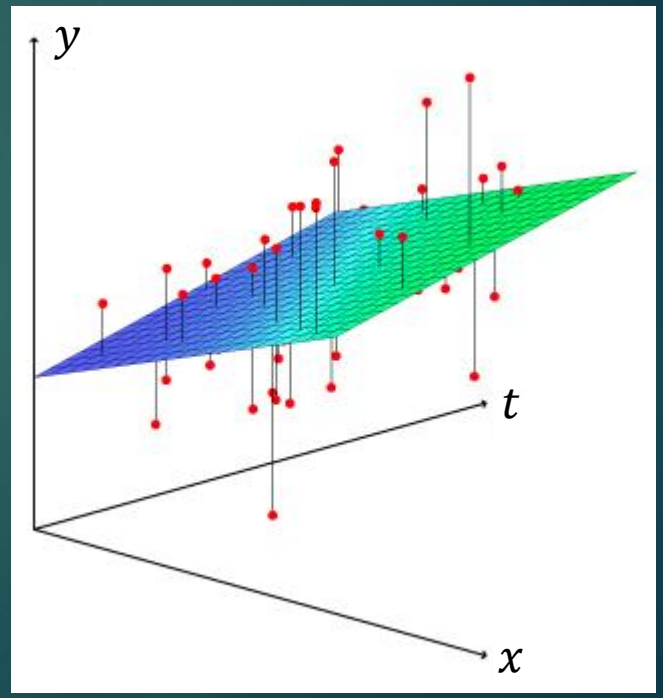
Finding the best-fitting straight line through the training examples

Linear Regression

- Multiple Linear Regression (多元線性回歸)
  - Handle multi-dimensional linear models
  - Using a hyperplane (超平面) ($y(x, t, z, \dots) = d + ax + bt + cz + \cdots$) to fit (multi-variate) data

$$\hat{y}(x, t, z, \dots) = \alpha_0 + \alpha_1 x + \alpha_2 t + \alpha_3 z + \cdots$$

    - Example: 2 independent variables
      - $\hat{y}(x, t) = \alpha_0 + \alpha_1 x + \alpha_2 t$ describe a plane



Linear Regression

- Basis Function Regression (基函數回歸)
  - Models complex (nonlinear) relationships between variables y and x using basis functions $f_i(x),\ i = 0,1,\dots,n-1$

$$\hat{y}(x) = \sum_{i=0}^{n-1} \alpha_i\, f_i(x) = \alpha_0 f_0(x) + \alpha_1 f_1(x) + \cdots + \alpha_{n-1} f_{n-1}(x)$$

    - $f_i(x)$'s could be nonlinear functions (transformations) of x
      - Effectively project one-dimensional x values into a higher dimension
    - $\hat{y}(x)$ still a *linear* model with respect to the parameters
    - Example: $\hat{y}(x) = \sum_{i=0}^{n-1} \alpha_i x^i = \alpha_0 + \alpha_1 x + \cdots + \alpha_{n-1} x^{n-1}$ becomes a polynomial regression (多項式回歸)

Least Squares Regression

- Derivation of Least Squares Solution
  - Enumerate the estimates $\hat{y}$ at each data point x_j

$$\hat{y}(x_1) = \alpha_0 f_0(x_1) + \alpha_1 f_1(x_1) + \cdots + \alpha_{n-1} f_{n-1}(x_1)$$
$$\hat{y}(x_2) = \alpha_0 f_0(x_2) + \alpha_1 f_1(x_2) + \cdots + \alpha_{n-1} f_{n-1}(x_2)$$
$$\vdots$$
$$\hat{y}(x_m) = \alpha_0 f_0(x_m) + \alpha_1 f_1(x_m) + \cdots + \alpha_{n-1} f_{n-1}(x_m)$$

    - In matrix form

$$\begin{bmatrix} \hat{y}(x_1) \\ \hat{y}(x_2) \\ \vdots \\ \hat{y}(x_m) \end{bmatrix} = \begin{bmatrix} f_0(x_1) & f_1(x_1) & \cdots & f_{n-1}(x_1) \\ f_0(x_2) & f_1(x_2) & \cdots & f_{n-1}(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_0(x_m) & f_1(x_m) & \cdots & f_{n-1}(x_m) \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_{n-1} \end{bmatrix}$$

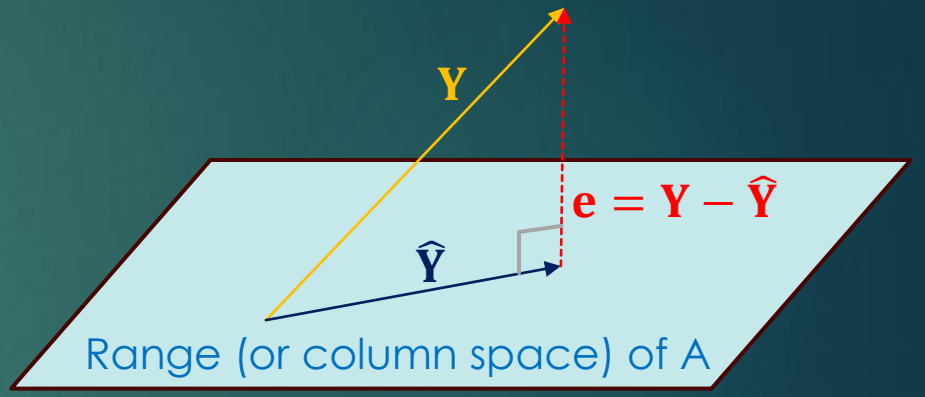
$\hat{\mathbf{Y}}$ $\quad$ $\mathbf{A}$ $\quad$ $\boldsymbol{\theta}$

$m \times 1$ $\quad$ $m \times n$ $\quad$ $n \times 1$

Least Squares Regression

- Let $\mathbf{Y} = [y_1, y_2, \ldots, y_m]^T$, then the error vector $\mathbf{e} = \mathbf{Y} - \widehat{\mathbf{Y}}$
  - The total squared error

$$E = \sum_{j=1}^{m} \left| y_j - \hat{y}(x_j) \right|^2 = \|\mathbf{e}\|^2 = \left\|\mathbf{Y} - \widehat{\mathbf{Y}}\right\|^2$$



- Minimum length of $\mathbf{e}$ achieved *if* the error vector $\mathbf{e}$ is *perpendicular to* the estimate $\widehat{\mathbf{Y}}$

(orthogonal) $\widehat{\mathbf{Y}}^T\mathbf{e} = 0 = \widehat{\mathbf{Y}}^T\left(\mathbf{Y} - \widehat{\mathbf{Y}}\right) = (\mathbf{A\theta})^T(\mathbf{Y} - \mathbf{A\theta}) = \boldsymbol{\theta}^T(\mathbf{A}^T\mathbf{Y} - \mathbf{A}^T\mathbf{A\theta})$

$\mathbf{A}^T\mathbf{A\theta} = \mathbf{A}^T\mathbf{Y}$

$\boldsymbol{\theta} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{Y}$ (Least Squares Solution)

$(\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$ is called the *pseudo-inverse* of $\mathbf{A}$
$(\mathbf{A}^T\mathbf{A})$ is invertible when $\mathbf{A}$ has linearly independent columns

Linear Least Squares (LLS) Regression in Python

LLS Regression in Python

- Consider the artificial data $y = 1 + x + xU(0,1)$ $(m = 100)$
  - x = np.linspace(0, 1, 100)
  - y = 1 + x + x * np.random.random(len(x))
- Simple linear regression with an estimation function

$$\hat{y}(x) = \alpha_1 x + \alpha_0$$

  - Theoretically, $\alpha_1 = 1.5$ and $\alpha_2 = 1.0$
- In Python, there are *many* different ways to conduct the least square regression
  - NumPy, SciPy, scikit-learn
  - statsmodels (統計模型)

Direct Inverse Method

- Example 1
  - Solve the optimum parameters using the direct inverse method according to the derived least squares solution

$$\boldsymbol{\theta} = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{Y}$$

    - Get the inverse of $(\mathbf{A}^T\mathbf{A})$ using numpy.linalg.inv()
  - Sample code: 10_1.py

Direct Inverse Method

```python
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-poster')

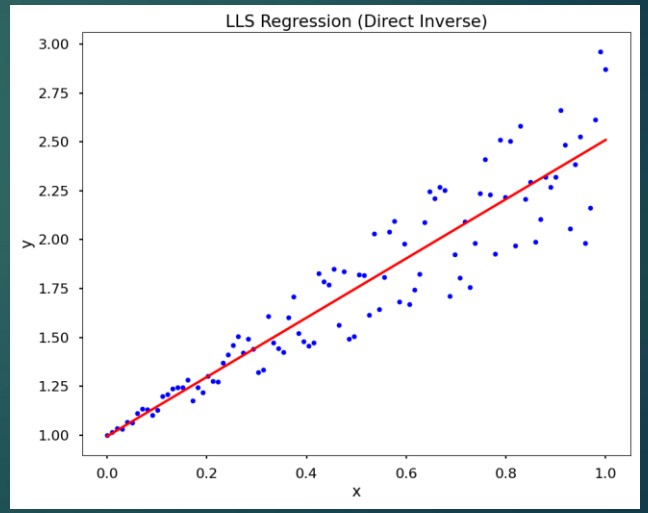
# generate x and y
x = np.linspace(0, 1, 100)
y = 1 + x + x * np.random.random(len(x))
# assemble matrix A
A = np.vstack([np.ones(len(x)), x]).T
# turn y into a column vector
y = y[:, np.newaxis]
print(y.shape)

# Direct least square regression
alpha = np.linalg.inv(A.T @ A) @ A.T @ y
print(alpha)
```

```
(100, 1)
```

```
[[0.99430995]
 [1.51582738]]
```

```python
# plot the results
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha[1]*x + alpha[0], 'r')
plt.title('LLS Regression (Direct Inverse)')
plt.xlabel('x')
plt.ylabel('y')
plt.tight_layout()
plt.show()
```



The Pseudo-inverse Method

- Example 2
  - Solve the optimum parameters using the pseudo-inverse method according to the derived least squares solution

$$\boldsymbol{\theta} = \begin{bmatrix} \alpha_0 \\ \alpha_1 \end{bmatrix} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{Y}$$

    - Get the pseudo-inverse $(\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$ using numpy.linalg.pinv()
  - Sample code: 10_2.py

The Pseudo-inverse Method

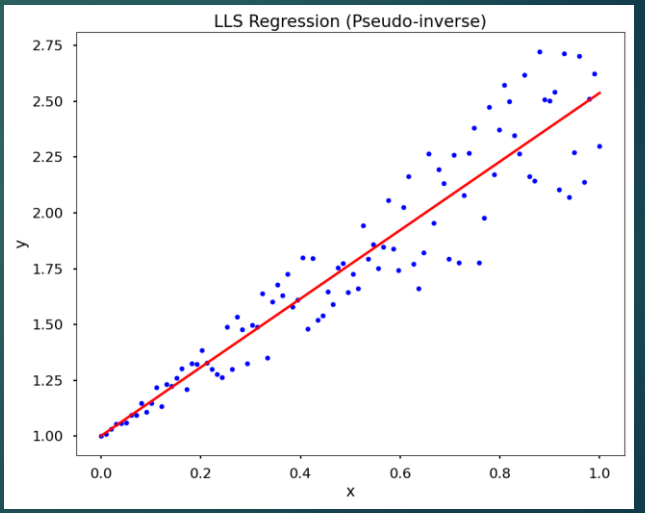
```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-poster')

# generate x and y
x = np.linspace(0, 1, 100)
y = 1 + x + x * np.random.random(len(x))
# assemble matrix A
A = np.vstack([np.ones(len(x)), x]).T
# turn y into a column vector
y = y[:, np.newaxis]
print(y.shape)

# Least square regression by pseudo-inverse
pinv = np.linalg.pinv(A)
alpha = pinv @ y
print(alpha)
```

```
[[1.00195876]
 [1.53499311]]
```

```
# plot the results
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha[1]*x + alpha[0], 'r')
plt.title('LLS Regression (Pseudo-inverse)')
plt.xlabel('x')
plt.ylabel('y')
plt.tight_layout()
plt.show()
```



NumPy's LLS Method

- Example 3
  - Solve the optimum parameters directly using numpy.linalg.lstsq()
  - Sample code: 10_3.py

NumPy's LLS Method

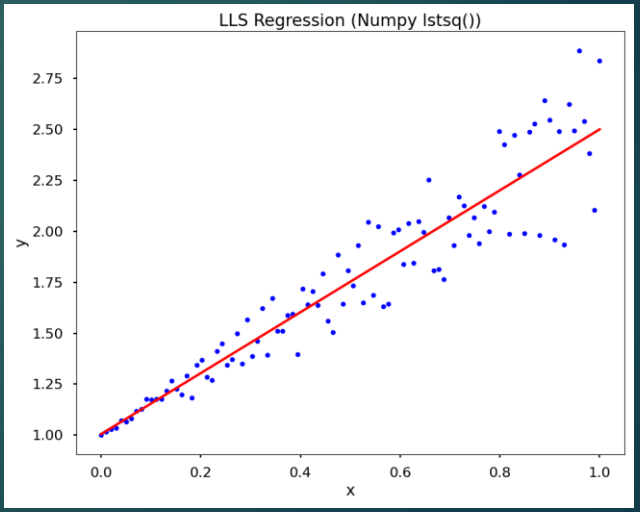
```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-poster')

# generate x and y
x = np.linspace(0, 1, 100)
y = 1 + x + x * np.random.random(len(x))
# assemble matrix A
A = np.vstack([np.ones(len(x)), x]).T
# turn y into a column vector
y = y[:, np.newaxis]
print(y.shape)

# Least square regression
alpha = np.linalg.lstsq(A, y, rcond=None)[0]
print(alpha)
```

```
[[1.00592029]
 [1.49263994]]
```

```
# plot the results
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha[1]*x + alpha[0], 'r')
plt.title('LLS Regression (Numpy lstsq())')
plt.xlabel('x')
plt.ylabel('y')
plt.tight_layout()
plt.show()
```



SciPy's Curve Fit Method

- Example 4
  - Solve the optimum parameters directly using scipy.optimize.curve_fit()
    - In fact, this SciPy function *can* even fit non-linear functions in the sense of the least squares

```
curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False,
check_finite=None, bounds=(-inf, inf), method=None, jac=None, *,
full_output=False, nan_policy=None, **kwargs) #
```

  - Sample code: 10_4.py

SciPy's Curve Fit Method

```
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-poster')

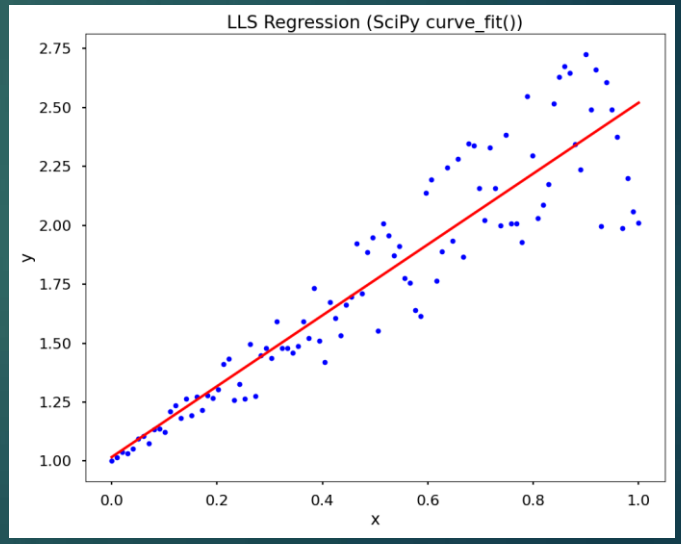
# generate x and y
x = np.linspace(0, 1, 100)
y = 1 + x + x * np.random.random(len(x))

# Curve Fit by a function
def func(x, a0, a1):
    y = a1*x + a0
    return y

alpha = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(alpha)
```

```
[1.01678655 1.50276192]
```

```
# plot the results
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha[1]*x + alpha[0], 'r')
plt.title('LLS Regression (SciPy curve_fit())')
plt.xlabel('x')
plt.ylabel('y')
plt.tight_layout()
plt.show()
```



LLS Regression in scikit-learn

- sklearn.linear_model
  - LinearRegression
  - Robust regression to outliers
    - RANSACRegressor
  - With Regularization
    - L2 penalty: Ridge
    - L1 penalty: Lasso
    - Weighted L1 + L2 penalty: ElasticNet

Use the same estimator API

```
>>> slr = LinearRegression()
>>> slr.fit(X, y)
>>> y_pred = slr.predict(X)
```

Polynomial Regression in scikit-learn

- Get features' high-order and interaction terms
  - sklearn.preprocessing.PolynomialFeatures

```
>>> import numpy as np
>>> from sklearn.preprocessing import PolynomialFeatures
>>> X = np.arange(6).reshape(3, 2)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> poly = PolynomialFeatures(2)
>>> poly.fit_transform(X)
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

Use the same transformer API

Use these features to fit a linear regressor

The features of X have been transformed from (X_1, X_2) to $(1, X_1, X_2, X_1^2, X_1X_2, X_2^2)$

Model Evaluation for Regression

- sklearn.metrics
  - mean squared error (MSE)
    - mean_squared_error
  - mean absolute error (MAE)
    - mean_absolute_error
  - coefficient of determination (R^2)
    - r2_score

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y^{(i)} - \hat{y}^{(i)}|$$

$$\begin{aligned} R^2 &= 1 - \frac{SSE}{SST} \\ &= 1 - \frac{\frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2}{\frac{1}{n}\sum_{i=1}^{n}(y^{(i)} - \mu_y)^2} \\ &= 1 - \frac{MSE}{Var(y)} \end{aligned}$$

```
mse_test = mean_squared_error(y_test, y_test_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)
```

Nonlinear Regression

Nonlinear Regression

- Linear regression requires that the estimation function be a linear combination of basis functions
- However, some functions *cannot* be put in this form
  - Example: Exponential functions

$$\hat{y}(x) = \alpha e^{\beta x}$$

  - Least squares regression is *still* appropriate
    - Either taking logarithms to transform the non-linear function into a linear function
    - Or using scipy.optimize.curve_fit() to estimate directly the parameters for the non-linear function using least squares

Log Tricks for Exponential Functions

- Take natural logarithms for both sides of exponential function $\hat{y}(x) = \alpha e^{\beta x}$

$$\tilde{y}(x) = \ln \hat{y}(x) = \ln \alpha + \beta x = \alpha_1 x + \alpha_0$$ (linear w.r.t. α_1, α_0)

  - Perform LLS regression on the linearized expression $\tilde{y}$ to find α_1, α_0
  - Recover the original parameters
    - $\alpha = e^{\alpha_0}$
    - $\beta = \alpha_1$

Log Tricks for Power Functions

- Log tricks can also be applied to power functions
- Take natural logarithms for both sides of $\hat{y}(x) = \alpha x^{\beta}$

$$\tilde{y}(x) = \ln \hat{y}(x) = \ln \alpha + \beta \ln x = \alpha_1 \ln x + \alpha_0$$ (linear w.r.t. α_1, α_0)

  - Perform LLS regression on the linearized expression $\tilde{y}$ to find α_1, α_0
  - Recover the original parameters
    - $\alpha = e^{\alpha_0}$
    - $\beta = \alpha_1$

SciPy's Curve Fit Function

- Use the scipy.optimize.curve_fit() function to fit *any* form function (exponential, power, …) and estimate the parameters of it

```
curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False,
check_finite=None, bounds=(-inf, inf), method=None, jac=None, *,
full_output=False, nan_policy=None, **kwargs)
```

Use non-linear least squares to fit a function, f, to data.

Assumes `ydata = f(xdata, *params) + eps`.

f : *callable*

The model function, f(x, …). It must take the independent variable as the first argument and the parameters to fit as separate remaining arguments.

Nonlinear Regression in Python

- Consider the artificial data $y = 0.1e^{0.3x} + 0.1\ U(-0.5,0.5)$ $(m = 100)$
  - x = np.linspace(0, 10, 100)
  - y = 0.1*np.exp(0.3*x) + 0.1*(np.random.random(len(x)) – 0.5)
- Nonlinear regression with an estimation function

$$\hat{y}(x) = \alpha e^{\beta x}$$

  - Theoretically, $\alpha = 0.1$ and $\beta = 0.3$
- Conduct the least square regression
  - Apply log tricks followed by using NumPy's LLS function
  - Use directly SciPy's curve fit function

Log Tricks + NumPy's LLS Method

- Example 5
  - Perform LLS regression using numpy.linalg.lstsq() on the linearized expression

$$\tilde{y}(x) = \ln \hat{y}(x) = \ln \alpha + \beta x = \alpha_1 x + \alpha_0$$

  - Sample code: 10_5.py

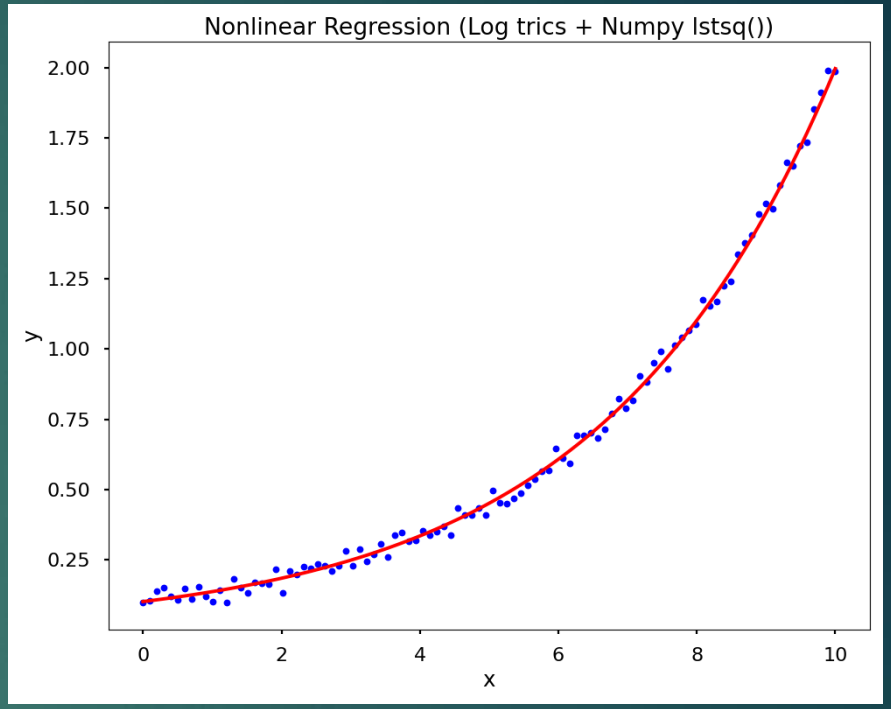
Log Tricks + NumPy's LLS Method

```
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-poster')

# generate x and y, and add some noise into y
x = np.linspace(0, 10, 100)
y = 0.1*np.exp(0.3*x) + 0.1*(np.random.random(len(x)) - 0.5)
# assemble matrix A
A = np.vstack([np.ones(len(x)), x]).T

# Least square regression on linearized expression
log_alpha, beta = np.linalg.lstsq(A, np.log(y), rcond = None)[0]
alpha = np.exp(log_alpha)
print(f'alpha={alpha}, beta={beta}')
```

```
# plot the results
plt.figure(figsize = (10,8))
plt.plot(x, y, 'b.')
plt.plot(x, alpha*np.exp(beta*x), 'r')
plt.title('Nonlinear Regression (Log trics + Numpy lstsq())')
plt.xlabel('x')
plt.ylabel('y')
plt.tight_layout()
plt.show()
```




```
alpha=0.10217484063715385, beta=0.297236438783891
```

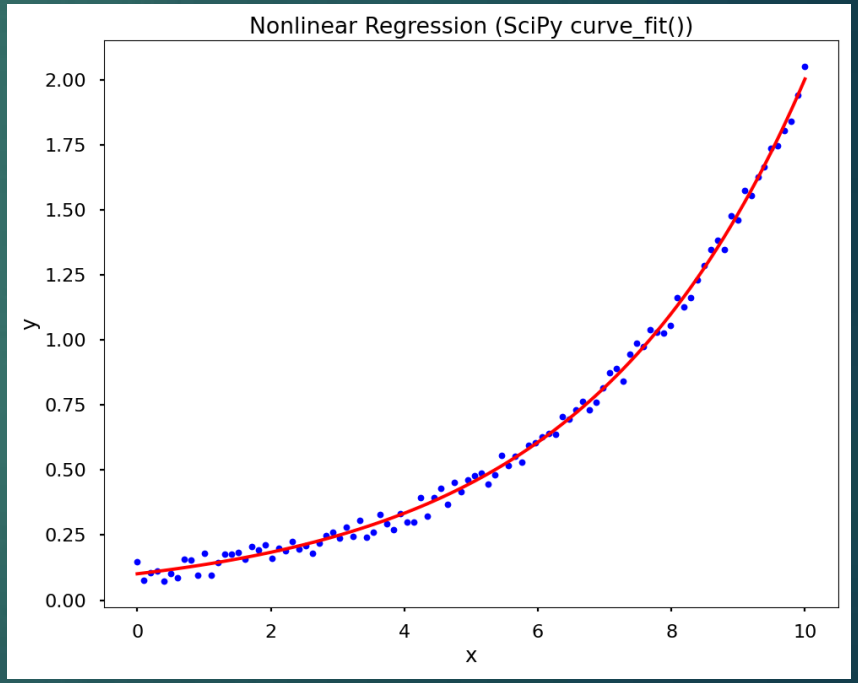
SciPy's Curve Fit Method

- Example 6
  - Solve the optimum parameters directly using scipy.optimize.curve_fit()
  - Sample code: 10_6.py

```
# Curve Fit by a function
def func(x, a, b):
    y = a*np.exp(b*x)
    return y

alpha, beta = optimize.curve_fit(func, xdata = x, ydata = y)[0]
print(f'alpha={alpha}, beta={beta}')
```

```
alpha=0.10128494414283319, beta=0.2985291836499794
```



Non-linear Regression using MLP

- Multilayer Perceptron (MLP) can also be used to perform non-linear regression
  - Passing input features through one or more hidden layers with non-linear activation functions (ReLU, sigmoid or tanh, etc.)
  - Loss Function: Typically trained using Mean Squared Error (MSE) to measure the distance between predictions and continuous targets

Non-linear Regression using MLP

- Example 7
  - Nonlinear regression using sklearn.neural_network.MLPRegressor
  - Sample code: 10_7.py

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

# 1. Generate synthetic non-linear data
np.random.seed(42)
X = np.linspace(-3, 3, 500).reshape(-1, 1)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])

# 2. Split and Scale features (MLPs are sensitive to feature scaling)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

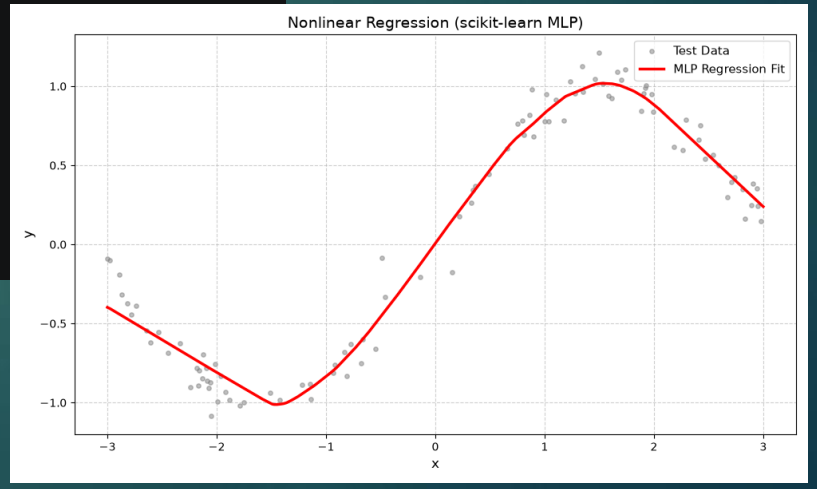
```python
# 3. Define and train the MLP Regressor
# We use two hidden layers (64 and 32 neurons)
# with ReLU activation
mlp = MLPRegressor(
    hidden_layer_sizes=(64, 32),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)
mlp.fit(X_train_scaled, y_train)

# 4. Predict
predictions = mlp.predict(X_test_scaled)
```

Non-linear Regression using MLP

```
34  # 5. Evaluation
35  r2score = r2_score(y_test, predictions)
36  print(f"Test Score (R²): {r2score:.4f}")
37
38  # plot the results
39  plt.figure(figsize=(10, 6))
40  plt.scatter(X_test, y_test, color='gray', alpha=0.5, label='Test Data', s=15)
41  X_all_scaled = scaler.transform(X)
42  y_all_pred = mlp.predict(X_all_scaled)
43  plt.plot(X, y_all_pred, color='red', linewidth=2.5, label='MLP Regression Fit')
44  plt.title('Nonlinear Regression (scikit-learn MLP)', fontsize=14)
45  plt.xlabel('x', fontsize=12)
46  plt.ylabel('y', fontsize=12)
47  plt.legend(fontsize=11)
48  plt.grid(True, linestyle='--', alpha=0.6)
49  plt.tight_layout()
50  plt.show()
```

```
Test Score (R²): 0.9763
```



You can use PyTorch to implement an MLP regressor that offers your maximum control over the network architecture, loss calculation and optimization process

Classwork

Assignment #6

- 目的：基準一年後糖尿病疾病進展之預測器
- 應用情境：
  - 臨床醫學定量評估 (Clinical Quantitative Assessment)
  - 假設病患的基礎生理特徵與化驗數據已由醫院預先採集完成
  - 本作業僅負責建立回歸模型來預測一年後的疾病進展指數

Assignment #6

- 資料集：sklearn.datasets.load_diabetes()
  - 442 筆 病患臨床紀錄
  - 10 維 生理與化驗特徵
    - 包含年齡、性別、BMI、血壓及 6 種血清化驗數據
  - 連續型目標值 (Target)：基準一年後疾病進展的定量測量值 (數值範圍介於 25 到 346 之間)

	age	sex	bmi	bp	s1	s2	s3	s4	s5	s6	target
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907	-0.017646	151.0
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332	-0.092204	75.0
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861	-0.025930	141.0
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688	-0.009362	206.0
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988	-0.046641	135.0

各項特徵已預先經過縮放 (每項特徵的平方和皆為 1)

Assignment #6

- 實作任務與要求：
  - 以三種不同的演算法建立回歸預測器並進行效能比較
  - 運用資料切分、特徵縮放與模型評估
    - 切分比例：80% 訓練集、20% 測試集
    - 模型評估：MSE 與 R^2
  - 模型要求：
    - 多元線性回歸：使用基礎線性模型作為效能 Baseline
    - 多項式回歸：以 PolynomialFeatures 設定 degree=2 進行特徵轉換
    - MLP 非線性回歸：以 PyTorch 實作 MLP regressor

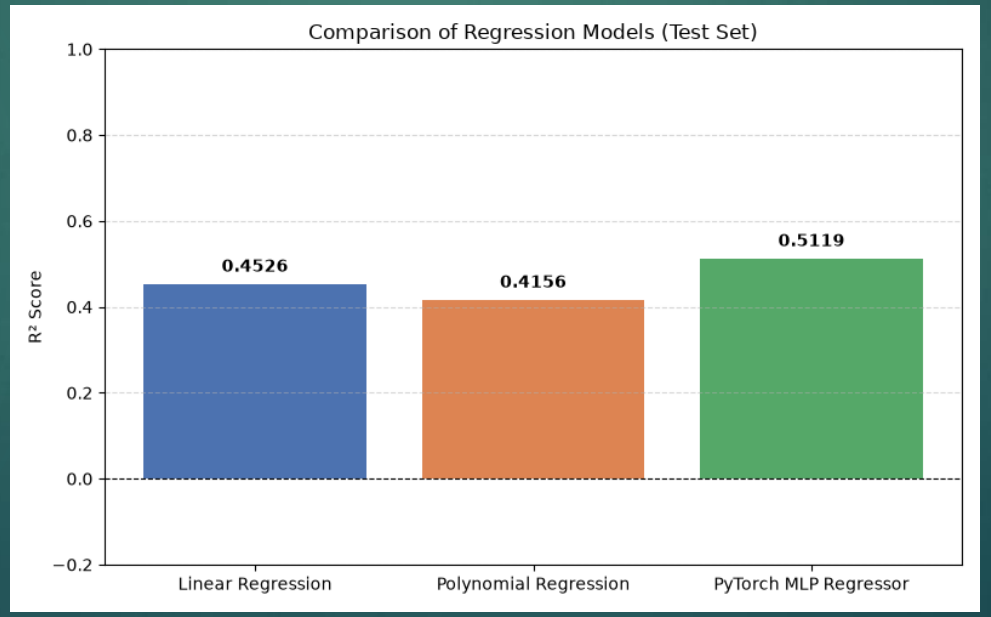
Assignment #6

- 預期結果 1
  - 輸出三種模型的測試集量化指標比較表
    - 分別計算並列出三種模型的 均方誤差 (MSE) 與 決定係數 (R^2 Score)
    - 在測試集 (約 89 筆樣本) 上三者之中最佳的 R^2 分數必須達 0.46 以上

```
========== 各模型效能比較 ==========
[Linear Regression] MSE: 2900.19 | R² Score: 0.4526
[Polynomial Regression] MSE: 3096.03 | R² Score: 0.4156
[PyTorch MLP Regressor] MSE: 2586.11 | R² Score: 0.5119
```

Assignment #6

- 預期結果 2
  - 視覺化呈現三種模型的效能對比繪製直方圖 (Bar Chart)
    - 直觀比較三種模型在測試集上的 R^2 Score
    - 標題須註明測試集字樣，長條上方須精準標示四位小數的 R^2 數值



在機器學習中，模型不是越酷炫、越複雜就越好

Assignment #6

- 此糖尿病資料集測試集 (Test Set) 的最佳 R^2 分數通常落在 0.45 ~ 0.55 之間

模型種類	R^2 區間	備註
多元線性回歸	**0.45 ~ 0.47**	基本盤，但容易受極端值影響
Lasso / Ridge 回歸	**0.47 ~ 0.50**	表現最穩定。透過 L1/L2 懲罰項能有效壓制共線性特徵
多項式回歸 (degree=2)	**0.30 ~ 0.42 (或負值)**	維度災難。不加正規化一定會因為特徵暴增而過擬合
KNN 回歸 (調參後)	**0.48 ~ 0.53**	透過調整適當的鄰居數 (如 $K = 9$)，在小資料集表現不錯
隨機森林 / XGBoost	**0.45 ~ 0.52**	樹模型擅長抓非線性，但在此資料集因樣本太少，很容易長出過深的樹導致過擬合
MLP / 類神經網路	**0.46 ~ 0.53**	架構必須非常小，且必須提早停止 (Early Stopping) 否則必過擬合

Thank you